

Where does all this waste come from?

Wolfgang Raschke^{1,*}, Massimiliano Zilli¹, Johannes Loinig², Reinhold Weiss¹,
Christian Steger¹ and Christian Kreiner¹

¹*Institute for Technical Informatics, Graz University of Technology, Austria*

²*NXP Semiconductors Austria, Gratkorn, Austria*

ABSTRACT

Agile development processes are more flexible than conventional ones. They emphasize iterative development and learning over feedback loops. Nevertheless, we experienced some pitfalls in the application of agile processes in dependable software systems. We present here the experiences we gathered in the construction of high-quality industrial software. Moreover, we will digest our experiences into a conceptual model of waste creation. This model will be refined to a case study where we take appropriate measurements in order to provide empirical evidence for it. Finally, we discuss the implications of the developed model, which helps to estimate the trade-off between agile and traditional software processes. Copyright © 2015 John Wiley & Sons, Ltd.

Received 17 March 2015; Accepted 20 May 2015

KEY WORDS: software process improvement; innovation; agile processes

1. INTRODUCTION

Today's dependable systems face the challenge of being built as safe, secure and reliable systems. The construction of such systems utilizes quality assurance methodologies such as code reviews, testing and static code analysis. Industry-relevant frameworks for process-based quality assurance exist for process maturity (SPICE [1]) and security (Common Criteria [2]). A higher capability level in such a framework implies more processes accomplished at a higher level of maturity. Generally, a higher level of assurance comes with a higher development effort. We investigated the development process of a highly secure software system. This project is developed with agile methodologies: Test-Driven Development [3], Continuous Integration [4] and Scrum [5]. These agile processes have considerable benefits in the industrial software project. Nevertheless, we observed some inconsistencies with regards to waste. Waste is an agile terminology, which denotes unnecessary work, which does not add to customer value [6]. We believe that it can be traced back to the agile way of working in the context of dependable systems. In order to build a suitable model for software development in dependable systems, we discuss a pattern of evolution, which has been observed in diverse industries. First, we apply this pattern to a general model for dependable systems. Second, we refine the dependable systems pattern in order to create a case study, which will give us the opportunity to provide evidence. We will discuss the implications which our model has on various levels of granularity and explain them regarding the agile development model and the V-model. Moreover, we show how our model can be used as framework of considering software development processes on a high level of abstraction. Finally, we conclude our findings and provide guidelines for scoping agile versus traditional processes.

*Correspondence to: Wolfgang Raschke, Institute for Technical Informatics, Graz University of Technology, Austria.

†E-mail: raschke.wolfgang@icloud.com

2. EXPERIENCE FROM AN INDUSTRIAL PROJECT

We gained experience from an industrial software project during the development of a highly secure system. It is imperative that evidence be provided for the purposes of security via certification. The so-called Common Criteria [2] is a documentation-based approach. To guarantee high levels of assurance, this evidence is the result of carefully designed and accomplished processes. A great amount of effort has been put into composing the appropriate documentation for these processes. We investigated the development process through interviews with software architects and developers. As a reference model, we utilized the V-model. Within the re-engineered V-model, we identified two agile iterative cycles. Our observations are outlined in Figure 1.

The inner cycle is an iterative loop that represents the agile development process that mainly focuses on the design and implementation of user stories. Because we are applying Test-Driven Development (TDD), writing tests belong to the code-and-test phase. Conversely, the execution of the tests is part of the unit test phase. The design comprises the specification of the component and its interface at component level. It is allowed to change interfaces and connections between components in this inner cycle. It pushes the outer architectural cycle. This is usually regarded as a bottom-up design methodology. The outer cycle represents the development of system requirements, architecture, integration tests and system verification (certification) in the V-model. As mentioned above, the outer cycle is driven by the inner cycle, which means that the architecture is first established incrementally at code and design level and then explicitly defined in the architecture phase. The system requirements do not evolve directly out of the inner circle, but the linkages between them and the architecture have to be permanently maintained. We found that there is a friction between both cycles, namely in the form of synchronizing implementation level artifacts (code and design) with architectural artifacts. Such a friction occurs in several manifestations which we describe in the following. First, changing interfaces affects testing. Thus, unit tests and integration tests have to be adopted according to the new interfaces. We will show this effect later in more detail. Second, changing interfaces have an impact on security evaluation because interface descriptions are part of the respective documentation. Third, some interfaces have to be known to link the components together because different programming languages are utilized. Hence, interfaces have to be reworked in the product integration, as well. Finally, we conclude that changing an interface causes rework in at least three obvious cases. The real cost cannot be measured reliably.

3. MODEL OF CHANGE IMPACT

In order to investigate potential causes for waste, we introduce a conceptual model of an innovation lifecycle. We apply the pattern described in [7] because it is simple and allows mapping to diverse circumstances.

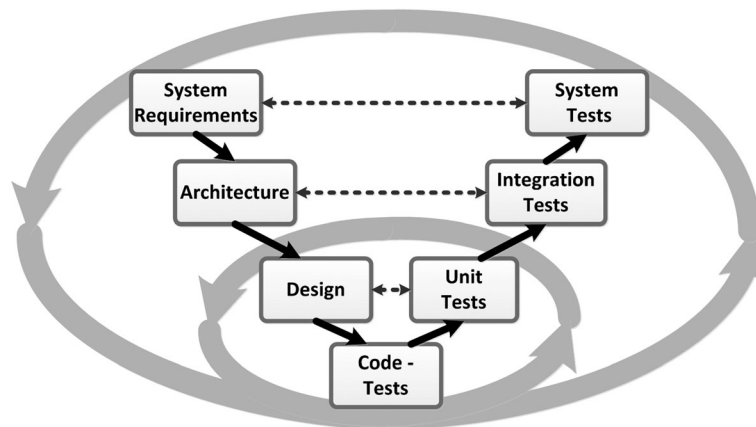


Figure 1. Iterative cycles in architecture and on component level.

3.1. Product innovation vs. process innovation

Abernathy [7] describes a lifecycle, where at the beginning of a new development, product innovation is accomplished at a high pace and process innovation on a relatively low one. Gradually, product innovation decreases its rate whereas process innovation increases. There is a lag between product innovation and process innovation. It is important to understand the meaning of product and process innovation, which we will clarify in the following.

3.2. Product innovation

Product innovation is the way a product is altered. Generally, product innovation is the change of what we build. Henderson [8] defines a model of innovation, where the rate of changing links between components is an important indicator. The change of links can be seen as the change of interfaces. In the following, we will use the term architectural evolution to label the change of interfaces.

3.3. Process innovation

Process innovation defines how we build products. This how usually includes all processes that embrace quality engineering, such as reviews, tests and static code analysis. It is likely that a change in an interface causes rework in the quality processes. A common practice is to increase the rate of process improvement when the interfaces are more or less stable [7].

3.4. Mapping to software terminology

In order to facilitate the construction of a hypothesis, which can be evaluated empirically in a software project, we map the previously outlined terminology to software engineering methodologies.

Product innovation is the sum of all activities that immediately affect a product. In a typical software project, these activities are: architecture, design and coding. We assume architectural evolution as a quantifiable indicator for product innovation. We define architectural evolution here as the rate of interface changes between at least two components. For example, in the architecture phase of the V-model, the rate of interface changes is high. In design and coding, this rate is, in comparison, low.

Process innovation is the aggregate of all activities that indirectly affect a product. In a software project these activities are usually: unit testing, integration testing, system testing, code review and static code analysis. In order to facilitate the interpretation we assume that process innovation is equal to all quality activities in a software project. The measurable quantity is the amount of quality activities at a certain point in time.

3.5. Definition of waste

Ikonen [6] defines waste as ‘basically everything that does not add to the customer value of a product’. This viewpoint regards everything except coding as waste, because it does not directly add value to the customer. This expectation is too narrow in our opinion, because testing and quality assurance add value indirectly. So, we regard waste as activities that neither *directly* nor *indirectly* add value.

3.6. Relation between waste, product and process artifacts

Our observations in the industrial case study suggest that waste is proportional to the evolution of the architecture in terms of changed interfaces dA . The definitions of the terms are given in Table I. The following formula states the generic model of waste creation:

$$W = \sum_i c_i \cdot dA = Q \cdot dA.$$

3.7. Evaluation with empirical data

The empirical evaluation has been conducted in cooperation with the company *NXP Semiconductors*. We evaluated a software project which is dedicated to the implementation of a highly secure embedded

Table I. Terminology of the generic model of waste creation.

Definition	Interpretation	Short
Architectural evolution	Change of component interfaces	dA
Waste	Effort with no additional value	W
Quality processes effort	Total effort for all quality activities	Q
Constant	Coupling factor no. i	c_i

system. About 100 developers participate at the project. The project includes several development teams, a testing team and a dedicated security team. The number of tests is typical for a high-quality embedded system of medium size: approximately 80 modules are tested by several thousand tests (unit tests, integration tests and acceptance tests). In order to apply the previously mentioned model to real data, we refine it in the following. We measured the number of failing test cases W . Additionally, we recorded the number of changed interfaces dA on a daily basis. We provide the explanation of the terms in Table II. Finally, we can construct a hypothesis, which can be evaluated with empirical data:

$$W_t = c_t \cdot dA.$$

Change of component interfaces: this is the accumulated number of changes in interfaces per day. This number has been measured on a daily basis, seven days a week. This information was retrieved from the source code repository.

Number of failing test cases: the number of failing test cases has been measured on a daily basis, seven days a week. The tests have been automatically executed each day on a continuous integration server. All test results have been logged.

We apply pre-processing to the data, which describes the changing of interfaces for the following reasons: First, there are remarkably few changing of interfaces on Saturdays and Sundays. Hence, there is a fundamental oscillation on a weekly basis. Second, the time span between altering an interface to the effect on the number of failing tests is not constant. This variation of time from cause to effect exists, because each and every interface is not immediately integrated into the tested product. In order to mitigate the mentioned effects, we apply a staged pre-processing to the recorded data, which is described in the following:

Step 0. Raw Data:

For the raw data, the Cross-Correlation Coefficient (CCC) equals 0.16.

Step 1. Constant Moving Average Filter:

In order to smooth the vector dA , we apply a simple moving average with a length of $N=7$:

$$dA_{avg}(n) = \frac{1}{N} \sum_{j=0}^{N-1} dA(n+j)$$

The CCC equals 0.51, and thus is remarkably higher, as before. We explain this by the filtering out of peaks that occur on weekends.

Table II. Terminology of the refined model which is evaluated empirically.

Definition	Interpretation	Short
Architectural evolution	Change of component interfaces	dA
Waste	Number of failing test cases	W
Constant	Coupling factor for testing	c_t

Step 2. Vector Norm:

Both vectors W and dA have considerably different scales. In order to apply vector normalization, we divide both vectors by their length and obtain the respective unit vectors.

$$dA_{norm} = \frac{dA_{avg}}{||dA_{avg}||}$$

$$W_{norm} = \frac{W_t}{||W_t||}$$

The normalization does not affect the Cross-Correlation Coefficient. The normalized signals are shown in Figure 2.

Step 3. Dynamic Time Warping:

In order to cope with the time variation, we apply *dynamic time warping* [9] [10], a non-linear signal processing algorithm. Dynamic time warping compares two signals and locally stretches and compresses the time axis in order to find an optimal alignment between both. A good alignment is characterized by a high similarity of both data series. The warping path p is a timely mapping between dA_{avg} and W_{norm} which is computed by the dynamic time warping algorithm:

$$p = dtw(dA_{avg}, W_{norm}).$$

The time mapping p allows the reconstruction of the signal dA_{rec} :

$$dA_{rec}(t) = dA_{norm}(p(t)).$$

The CCC equals 0.87 and shows that there is a high correlation between the pre-processed data. The reconstructed signal dA_{rec} and the reference signal W_{norm} now show a remarkable visual similarity, as can be seen in Figure 3. Thus, we can assume a proportional relation between both signals.

4. IMPLICATIONS

In the presented study, there is correlation between the evolution of the architecture and failing test cases. This undermines the general hypothesis that the evolution of the architecture causes

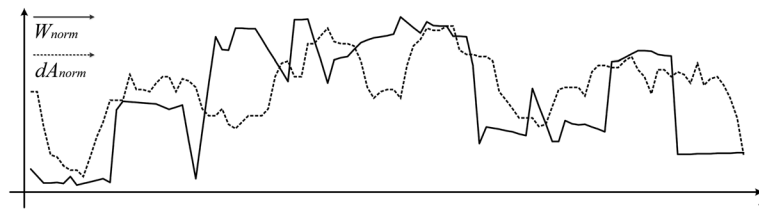


Figure 2. W_{norm} and dA_{norm} after the normalization.

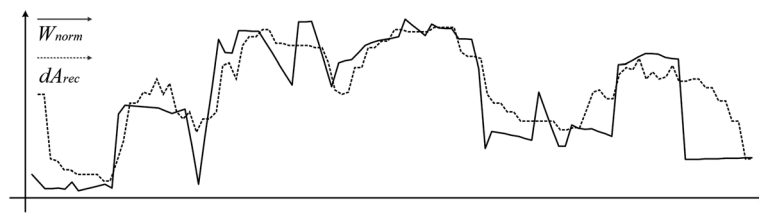


Figure 3. After data processing step 3, the signals W_{norm} and dA_{rec} are obviously similar.

rework in quality processes. The implications can be applied to the V-model and agile development processes.

4.1. Implications for the V-model

The V-model reflects Abernathy's pattern of innovation [7] where in a cycle of experimentation, learning and refinement, an architecture is defined. In this architecture phase, there are a high amount of changing interfaces (see dotted line in Figure 4 a) but quality processes are not unfolded to full maturity (see continuous line in Figure 4 a). So, the product of both curves is in the middle range (see Figure 4 b).

Certainly, there occurs a point in time, where architectural activities slow down. In the V-model this is usually the phase of implementation, which is the pivotal element of the V, and thus has connections to architectural activities and to quality processes. In this phase, architecture and quality processes are assumed to operate at a medium velocity. The product of both is high in this phase. In the third phase, the interfaces are mostly stable and quality assurance is at its highest level of the whole lifecycle. The product of both curves is in the middle range.

For a better illustration of the curves in Figure 4, we apply sample values in Table III. The sum of a column represents the architectural evolution (dA), effort for quality processes (Q) and the resulting total waste (W) of the model.

4.2. Implications for the agile model

In agile processes, the sequence of process steps is abandoned. Rather, the activities are accomplished concurrently in iterative cycles. In such an iterative cycle, each activity (architecting, coding, testing and verification) has to be performed. We assume for architectural evolution (see dotted line in Figure 5 a) and for quality activities (see continuous line in Figure 5 a) a constant and medium velocity. As can be seen in Figure 5 b, the product of both is constantly very high. Therefore, the interface changes create huge effort because all quality activities in the current iteration are immediately affected.

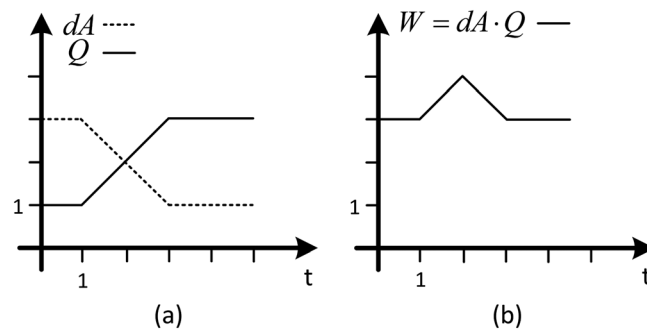


Figure 4. Coherence between waste W , changing interfaces dA and total effort for quality processes Q in the V-model.

Table III. Sample data for the model of waste creation for the V-model and agile processes.

Time	V-model dA	V-model Q	V-model W	Agile dA	Agile Q	Agile W
0	3	1	3	2	2	4
1	3	1	3	2	2	4
2	2	2	4	2	2	4
3	1	3	3	2	2	4
4	1	3	3	2	2	4
Sum	10	10	16	10	10	20

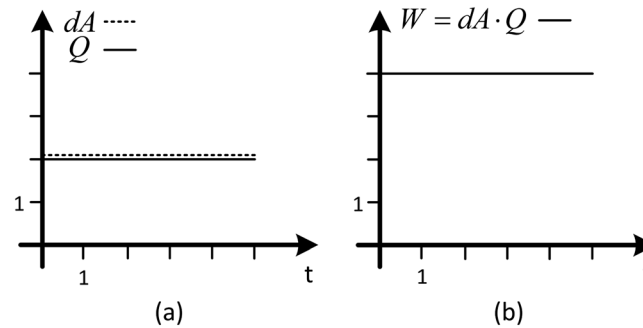


Figure 5. Coherence between waste W , changing interfaces dA and total effort for quality processes Q in the agile model.

For a better illustration of the curves in Figure 5, we apply sample values in Table III. The sum of a column represents the agile evolution (dA), the agile effort for quality processes (Q) and the resulting total waste (W) of the model.

4.3. Conclusion

In Table III, the effort for architecture and processes is equal (Sum = 10). This allows a comparison of the resulting waste: for agile working practices, the model predicts a higher resulting waste compared to the V-model. Of course, the presented model of waste creation is an abstraction. In reality there is some amount of approximation in the application of such a model. Nevertheless, the theoretical model is suitable for deliberation. Such deliberation suggests the following conclusion: agile methods are more appropriate for software projects with low demand for quality and possibly only acceptance tests. Traditional processes, like the V-model process is more appropriate in software projects with a high demand for quality and respective activities.

ACKNOWLEDGEMENTS

- Project partners are NXP Semiconductors Austria GmbH and TU Graz. The project is funded by the Austrian Federal Ministry for Transport, Innovation and Technology under the FIT-IT contract FFG 832171. The authors would like to thank pure systems GmbH for support.

REFERENCES

1. Messnarz R et al. Integrated automotive SPICE and safety assessments. *Software Process: Improvement and Practice* 2009; **14**(5):279288. DOI: 10.1002/spip.429.
2. Common criteria for information technology security evaluation—part 1–3. <https://www.commoncriteriaportal.org/cc/> [16 March 2015].
3. Janzen D, Saiedian H. Test-driven development: concepts, taxonomy, and future direction. *Computer* 2009; **38**(9):43–50. DOI: 10.1109/MC.2005.314.
4. Duvall P, Matyas S, Glover A. *Continuous integration: improving software quality and reducing risk*. Pearson Education: Boston, 2007.
5. Cohn M. *Succeeding With Agile*. Pearson Education: Boston, 2009.
6. Ikonen M, Kettunen P, Oza N. Exploring the sources of waste in Kanban software development projects. In *Proceedings of the 36th EUROMICRO Conference on Software Engineering and Advanced Applications*, 2010. DOI: 10.1109/SEAA.2010.40.
7. Abernathy WJ, Utterback JM. Patterns of industrial innovation. *Technology Review* 1978; **80**(7):41–47.
8. Henderson R, Clark K. Architectural innovation: the reconfiguration of existing product technologies and the failure of established firms. *Administrative Science Quarterly* 1990; **35**(1):9–30.
9. Giorgino T. Computing and visualizing dynamic time warping alignments in R: the dtw package. *Journal of Statistical Software* 2009; **31**(7):1–24.
10. Müller M. *Dynamic Time Warping. Information Retrieval for Music and Motion*. Springer: Berlin Heidelberg, 2007. DOI: 10.1007/978-3-540-74048-34.